

two of the most complex challenges in post-silicon validation. In particular, root cause for a failure observed on a specific design component can be in a completely different part of the design. One of the first challenges is to determine whether the bug is a silicon issue or a problem with design logic. If it is determined to be a logic error, the goal is typically to recreate it on a pre-silicon platform (such as RTL simulation and FPGA). The exact post-silicon scenario cannot be exercised in a pre-silicon platform. One second of silicon execution takes several weeks or months to exercise on RTL simulation.

Consequently, bulk of the creative effort in post-silicon creates a scenario that exhibits the same behaviour as the original post-silicon failure but involves an execution small enough to be replayable in pre-silicon platforms.

In addition to this key effort, other activities for bug resolution include grouping and validating the bug fix. The same design error might result in different observable failures for different tests. For example, a deadlock in a protocol might result in a system crash in one test and a hang in another.

Given aggressive validation schedules, it is imperative not to waste resources to debug the same error twice. Consequently, it is critical to group together errors arising from the same root cause. This is a highly non-trivial exercise. One must bucket errors with the same (or similar) root cause but with possibly different observable failures before analysing these. Finally, once a fix has been developed, one must validate the fix itself to ensure that neither does it correct the original error nor introduce a new one.

Given the scope and complexity of post-silicon validation and the aggressive schedule under which it must be performed, it is clear that it needs meticulous planning. Other requirements include defining post-silicon tests, test cards, custom boards and more. In fact, a crucial activity during the pre-silicon time frame is post-silicon readiness, that is, activities geared towards streamlined execution of post-silicon validation. Post-silicon readiness activities proceed concurrently with system architecture,



Fig. 17: Different validation challenges

design, implementation and pre-silicon validation.

Test plans. These constitute arguably the most critical and fundamental readiness activity for post-silicon validation. The objective is to identify different coverage targets, corner cases and functionalities that need to be tested for the system being deployed.

Post-silicon test plans are typically more elaborate than pre-silicon plans, since these often target system-level use-cases of the design that cannot be exercised during pre-silicon validation. Test plan development starts concurrently with design planning. When the test plan development starts, a detailed design (or even an elaborate microarchitecture for the most part) is unavailable.

Initial test planning, correspondingly, depends on high-level architectural specifications. As the design matures and more and more design features are developed, test plans undergo refinement to account for these features. The plans also need

to account for target applications, the new versus legacy IPs used in the system design and so on.

On-chip instrumentation

On-chip instrumentation refers to the DfD features integrated into the silicon to facilitate post-silicon debug and validation. A key target of DfD is observability. Modern SoC designs include a significant amount of hardware for this purpose, with estimates running up to 20 per cent or more in silicon real estate in some cases.

Two critical observability features are scan chains and signal tracing. Scan chains enable observability of the internal state of design. These are highly-mature architectures originally developed for identifying manufacturing defects in the circuit. However, these also provide critical observability during post-silicon validation.

Signal tracing, on the other hand, specifically targets post-silicon validation. The objective is to identify a small set (typically 100s) of internal